

Lottoziehung in Python

Dieses Dokument erklärt Schritt für Schritt, wie das Programm `lotto.py` funktioniert. Es zieht 6 aus 45 Zahlen ohne Wiederholung und erstellt anschließend eine Statistik über viele Ziehungen.

1. Die Aufgabenstellung

Laut Angabe gibt es zwei wichtige Bedingungen: Keine Zahl darf zweimal gezogen werden, und der Zufallszahlengenerator darf pro gezogener Zahl nur **ein einziges Mal** aufgerufen werden. Die naheliegende Lösung ("Zahl ziehen, und falls sie schon da ist, nochmal ziehen") ist deshalb nicht erlaubt, weil man dabei unter Umständen sehr oft neu ziehen müsste.

2. Die Methode `lottoziehung()`

```
def lottoziehung(anzahl=6, max_zahl=45):
    zahlen = list(range(1, max_zahl + 1))
    gezogen = []
    for i in range(anzahl):
        letzte = max_zahl - 1 - i
        idx = random.randint(0, letzte)
        gezogen.append(zahlen[idx])
        zahlen[idx], zahlen[letzte] = zahlen[letzte], zahlen[idx]
    return gezogen
```

Zuerst wird eine Liste mit allen Zahlen von 1 bis 45 angelegt. Die Variable `letzte` zeigt auf das Ende des Bereichs, aus dem noch gezogen werden darf. Am Anfang ist das Index 44, nach jeder Ziehung wird dieser Bereich um eins kleiner.

In jedem Durchlauf wird genau eine Zufallszahl erzeugt, nämlich ein Index zwischen 0 und `letzte`. Die Zahl an dieser Stelle wird gezogen. Danach kommt der entscheidende Trick: Die gezogene Zahl wird mit der Zahl an der Position `letzte` vertauscht. Damit liegt sie außerhalb des Bereichs, aus dem noch gezogen wird, und kann nie wieder getroffen werden.

Dieses Verfahren ist als **Fisher-Yates-Verfahren** bekannt. Für alle 45 Zahlen bräuchte man also genau 45 Aufrufe von `random.randint()`, was genau der Vorgabe entspricht.

Beispiel mit nur 5 Zahlen

Zur Veranschaulichung ein kleines Beispiel, bei dem 3 aus 5 Zahlen gezogen werden:

Schritt	Bereich	Zufallsindex	gezogen	Liste danach
Start	-	-	-	[1, 2, 3, 4, 5]
1	Index 0-4	1	2	[1, 5, 3, 4 2]
2	Index 0-3	0	1	[4, 5, 3 1, 2]
3	Index 0-2	2	3	[4, 5 3, 1, 2]

Der senkrechte Strich trennt den noch freien Bereich (links) von den bereits gezogenen Zahlen (rechts). Gezogen wurden 2, 1 und 3, und zwar mit genau drei Zufallszahlen.

3. Die Methode `statistik()`

```
def statistik(ziehung, stat):
    for zahl in ziehung:
        stat[zahl] += 1
```

Diese Methode bekommt eine einzelne Ziehung (also 6 Zahlen) und das Statistik-Dictionary übergeben. Für jede gezogene Zahl wird der passende Zähler um eins erhöht. Da ein Dictionary in Python veränderbar ist, wirkt sich die Änderung direkt auf das Original aus, ein `return` ist nicht nötig.

4. Die Methode `lotto_statistik()`

```
def lotto_statistik(anzahl_ziehungen, max_zahl=45):
    stat = {zahl: 0 for zahl in range(1, max_zahl + 1)}
    for _ in range(anzahl_ziehungen):
        ziehung = lottoziehung()
        statistik(ziehung, stat)
    return stat
```

Hier wird zuerst ein Dictionary erstellt, in dem jede Zahl von 1 bis 45 als Schlüssel mit dem Startwert 0 steht. Danach werden so viele Ziehungen durchgeführt wie gewünscht, und nach **jeder** Ziehung wird die Statistikmethode aufgerufen. Das Unterstrich-Zeichen in der Schleife zeigt an, dass die Zählvariable selbst nicht gebraucht wird.

5. Das Hauptprogramm

```
if __name__ == "__main__":
    print("Lottoziehung:", sorted(lottoziehung()))
    for n in (1000, 10000, 100000):
        stat = lotto_statistik(n)
    ...
```

Zuerst wird eine einzelne Ziehung sortiert ausgegeben. Anschließend wird die Statistik für 1000, 10000 und 100000 Ziehungen berechnet. Für jede Zahl werden die absolute Häufigkeit und der Anteil in Prozent ausgegeben.

6. Was man in der Statistik erwarten kann

Bei jeder Ziehung werden 6 von 45 Zahlen gezogen. Jede Zahl hat also eine Wahrscheinlichkeit von $\frac{6}{45}$, das sind ungefähr 13,33 %, pro Ziehung dabei zu sein. Bei wenigen Ziehungen schwanken die Werte noch deutlich, mit steigender Anzahl nähern sich alle Zahlen immer mehr diesem Wert an. Das ist das **Gesetz der großen Zahlen** und zeigt gleichzeitig, dass der Algorithmus keine Zahl bevorzugt.

Hinweis zur Angabe

Die in der Angabe genannte Funktion `random.getrand()` gibt es in Python nicht. Verwendet wurde stattdessen `random.randint(a, b)`, die eine ganze Zufallszahl zwischen a und b (beide eingeschlossen) liefert.